

MovieLens Rating Prediction

HarvardX PH125.9x Capstone Project

Romain

August 25, 2026

Contents

1	Introduction	1
2	Methods and Analysis	2
2.1	Creating the datasets	2
2.2	A first look at the data	3
2.3	Setting up an internal train/test split	4
2.4	Stage 1: mean and bias terms	4
2.5	Stage 2: regularization	6
2.6	Stage 3: matrix factorization	8
3	Results	9
4	Conclusion	10

1 Introduction

This report covers my work on the MovieLens capstone: building a model that predicts what rating a user would give a movie, using the 10M-rating version of the MovieLens dataset from GroupLens. The dataset has about 10 million ratings spread across roughly 70,000 users and 10,700 movies, which is small enough to work with on a laptop but big enough that some of the tricks from the earlier courses in this series (small toy datasets, exact cross-validation on the full thing) stop being practical.

The task itself is straightforward to state: split the data into a set I get to train on (`edx`) and a set I only touch once at the very end (`final_holdout_test`), then minimize RMSE between predicted and actual ratings on that final set. What's less straightforward is getting there without accidentally leaking information from the test set into model choices along the way, which is really the whole point of this exercise.

My approach went through three stages. I started with the mean-plus-bias-terms model that's taught in the machine learning course of this series (movie effect, then user effect, then a regularized version of both) - mostly to have a working baseline and to make sure I understood what was going on before adding complexity. That approach tops out around an RMSE of 0.865, which is decent but not great. I then moved to matrix factorization using the `recosystem` package, which models each user and movie as a vector in some shared latent space and learns those vectors by minimizing reconstruction error over the observed ratings. That got the RMSE down to about **0.779** on the final holdout set, which is where I landed for the actual submission.

I'll walk through the data prep, then the exploration I did to get a feel for the dataset, then each modeling stage in order, and finish with the results and what I'd try next if I had more time (or more CPU cores - more on that later).

2 Methods and Analysis

2.1 Creating the datasets

The code below is the one provided for the project - it downloads the MovieLens 10M zip file, builds a single ratings/movies table, and splits off 10% as `final_holdout_test`, making sure that any user or movie appearing in the holdout set also appears in `edx` (otherwise you'd have no way to predict for them). I haven't changed anything here; I'm including it as-is since the exact split matters for reproducibility.

```
options(timeout = 120)

dl <- "ml-10M100K.zip"
if(!file.exists(dl))
  download.file("https://files.grouplens.org/datasets/movielens/ml-10m.zip", dl)

ratings_file <- "ml-10M100K/ratings.dat"
if(!file.exists(ratings_file))
  unzip(dl, ratings_file)

movies_file <- "ml-10M100K/movies.dat"
if(!file.exists(movies_file))
  unzip(dl, movies_file)

ratings <- as.data.frame(str_split(read_lines(ratings_file), fixed("::")), simplify = TRUE),
                      stringsAsFactors = FALSE)
colnames(ratings) <- c("userId", "movieId", "rating", "timestamp")
ratings <- ratings %>%
  mutate(userId = as.integer(userId),
         movieId = as.integer(movieId),
         rating = as.numeric(rating),
         timestamp = as.integer(timestamp))

movies <- as.data.frame(str_split(read_lines(movies_file), fixed("::")), simplify = TRUE),
                      stringsAsFactors = FALSE)
colnames(movies) <- c("movieId", "title", "genres")
movies <- movies %>%
  mutate(movieId = as.integer(movieId))

movielens <- left_join(ratings, movies, by = "movieId")

set.seed(1, sample.kind = "Rounding")
test_index <- createDataPartition(y = movielens$rating, times = 1, p = 0.1, list = FALSE)
edx <- movielens[-test_index,]
temp <- movielens[test_index,]

final_holdout_test <- temp %>%
  semi_join(edx, by = "movieId") %>%
  semi_join(edx, by = "userId")

removed <- anti_join(temp, final_holdout_test)
edx <- rbind(edx, removed)

rm(dl, ratings, movies, test_index, temp, movielens, removed)
```

2.2 A first look at the data

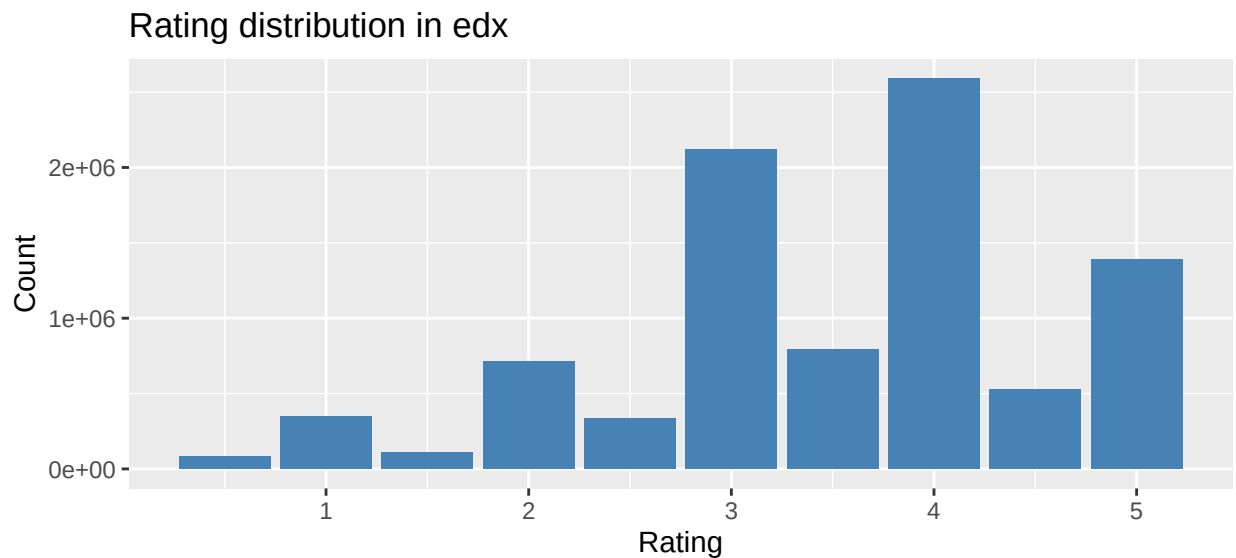
Before building anything, I wanted a basic sense of what I was working with. `edx` ends up with 9000055 rows and 6 columns, covering 10677 distinct movies and 69878 distinct users.

```
edx %>%  
  summarize(n_ratings = n(),  
            n_movies = n_distinct(movieId),  
            n_users = n_distinct(userId))
```

n_ratings	n_movies	n_users
9000055	10677	69878

A couple of things stood out right away. First, ratings aren't evenly distributed - whole-star ratings are a lot more common than half-star ones:

```
edx %>%  
  count(rating) %>%  
  ggplot(aes(x = rating, y = n)) +  
  geom_col(fill = "steelblue") +  
  labs(x = "Rating", y = "Count", title = "Rating distribution in edx")
```

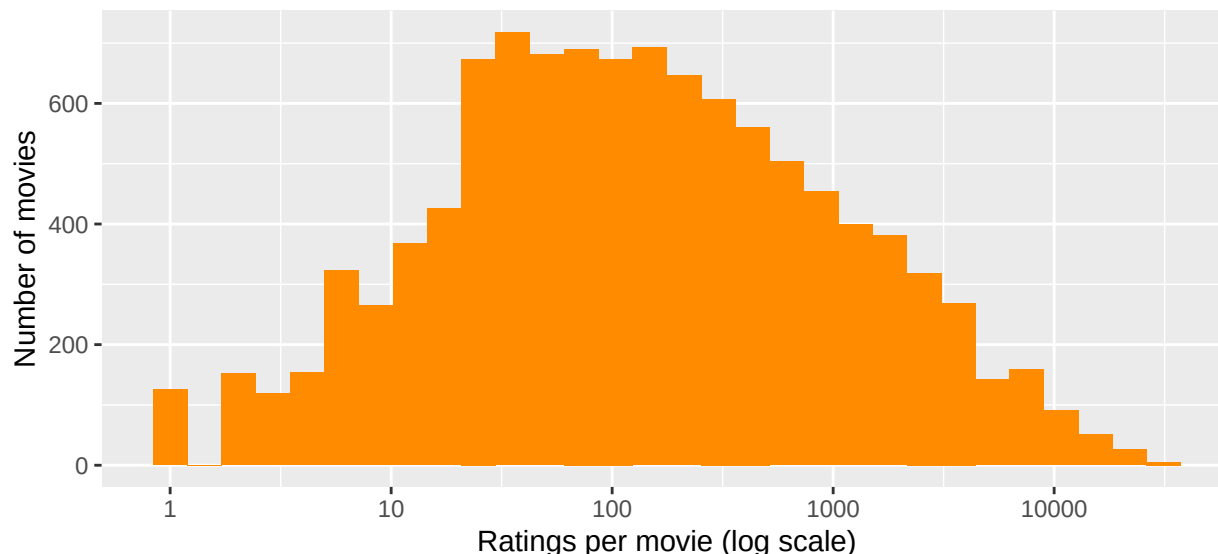


4, 3 and 5 stars alone account for most of the ratings, and half-star ratings (1843170 of them, versus 7156885 whole-star ratings) are clearly a secondary option users reach for less often - which makes sense since they were added to the platform later than whole-star ratings.

Second, both movies and users vary a lot in how many ratings they have:

```
p1 <- edx %>%  
  count(movieId) %>%  
  ggplot(aes(n)) +  
  geom_histogram(bins = 30, fill = "darkorange") +  
  scale_x_log10() +  
  labs(x = "Ratings per movie (log scale)", y = "Number of movies")
```

p1



Some movies (the popular ones - Pulp Fiction leads the pack with 31362 ratings) have tens of thousands of ratings, while plenty of others have just a handful. Same story for users: some have rated thousands of movies, most have rated a few dozen. This is exactly the kind of unevenness that regularization is meant to deal with, so I kept it in mind going into the modeling.

2.3 Setting up an internal train/test split

The project instructions are explicit that `final_holdout_test` can only be used once, at the very end, for the final RMSE - not for picking between models or tuning parameters. So before doing anything else I split `edx` itself into a training set and a smaller internal test set, using the same logic as the official split (80/20 this time), and did all my model comparisons against that.

```
RMSE <- function(true_ratings, predicted_ratings) {
  sqrt(mean((true_ratings - predicted_ratings)^2))
}
```

```
set.seed(1, sample.kind = "Rounding")
test_index <- createDataPartition(y = edx$rating, times = 1, p = 0.2, list = FALSE)
train_set <- edx[-test_index,]
temp <- edx[test_index,]

test_set <- temp %>%
  semi_join(train_set, by = "movieId") %>%
  semi_join(train_set, by = "userId")

removed <- anti_join(temp, test_set)
train_set <- rbind(train_set, removed)

rm(test_index, temp, removed)
```

2.4 Stage 1: mean and bias terms

I started simple, mostly to establish a baseline and sanity-check that everything downstream was working as expected. Predicting just the average rating for everything gives a rough sense of the “floor”:

```
mu <- mean(train_set$rating)

rmse_results <- tibble(method = "Just the average", RMSE = RMSE(test_set$rating, mu))
rmse_results
```

method	RMSE
Just the average	1.059904

Adding a movie-specific effect (b_i , how much better or worse a given movie tends to be rated than average) helps a fair amount, since some movies really are just rated higher than others regardless of who's watching:

```
movie_avgs <- train_set %>%
  group_by(movieId) %>%
  summarize(b_i = mean(rating - mu))

pred_movie <- mu + test_set %>%
  left_join(movie_avgs, by = "movieId") %>%
  pull(b_i)

rmse_results <- rmse_results %>%
  add_row(method = "Movie effect", RMSE = RMSE(test_set$rating, pred_movie))
rmse_results
```

method	RMSE
Just the average	1.0599043
Movie effect	0.9437429

Adding a user-specific effect (b_u , how harsh or generous a given user tends to be) on top of that helps again:

```
user_avgs <- train_set %>%
  left_join(movie_avgs, by = "movieId") %>%
  group_by(userId) %>%
  summarize(b_u = mean(rating - mu - b_i))

pred_movie_user <- test_set %>%
  left_join(movie_avgs, by = "movieId") %>%
  left_join(user_avgs, by = "userId") %>%
  mutate(pred = mu + b_i + b_u) %>%
  pull(pred)

rmse_results <- rmse_results %>%
  add_row(method = "Movie + user effect", RMSE = RMSE(test_set$rating, pred_movie_user))
rmse_results
```

method	RMSE
Just the average	1.0599043
Movie effect	0.9437429
Movie + user effect	0.8659319

2.5 Stage 2: regularization

The bias terms above are just averages, which means movies or users with very few ratings get noisy, unreliable estimates - a movie rated once at 5 stars gets a b_i as if it were a genuinely great movie, when really that's a sample size of one. Regularization shrinks these estimates toward zero in proportion to how few ratings they're based on, controlled by a penalty λ :

$$\hat{b}_i(\lambda) = \frac{1}{n_i + \lambda} \sum_u (Y_{u,i} - \hat{\mu})$$

I picked λ by trying a range of values and checking which one minimizes RMSE on `test_set`:

```
lambdas <- seq(0, 10, 0.25)

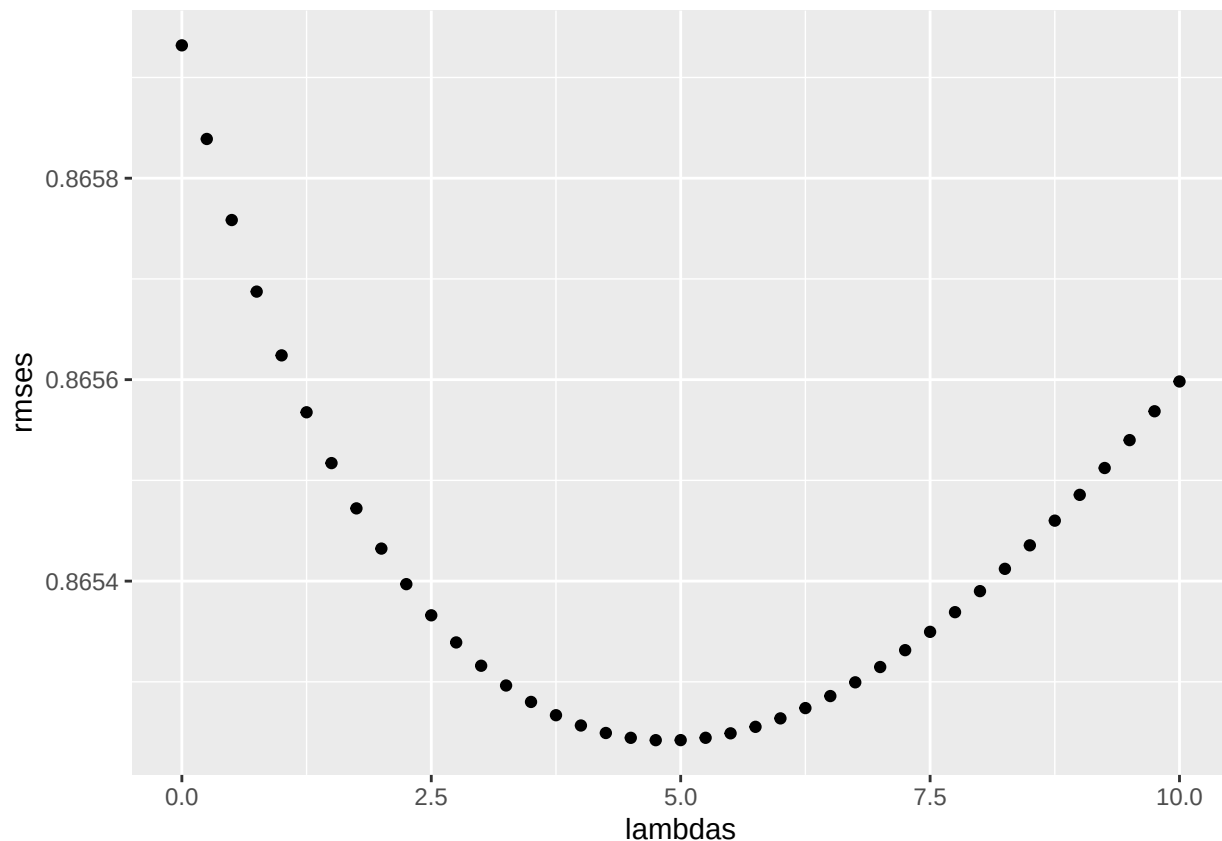
rmses <- sapply(lambdas, function(l) {
  b_i <- train_set %>%
    group_by(movieId) %>%
    summarize(b_i = sum(rating - mu) / (n() + l))

  b_u <- train_set %>%
    left_join(b_i, by = "movieId") %>%
    group_by(userId) %>%
    summarize(b_u = sum(rating - b_i - mu) / (n() + l))

  pred <- test_set %>%
    left_join(b_i, by = "movieId") %>%
    left_join(b_u, by = "userId") %>%
    mutate(pred = mu + b_i + b_u) %>%
    pull(pred)

  result <- RMSE(test_set$rating, pred)
  rm(b_i, b_u, pred); gc(verbose = FALSE) # this loop was chewing through
                                          # memory on my machine otherwise
  result
})

qplot(lambdas, rmses)
```



```
lambda_opt <- lambdas[which.min(rmses)]
lambda_opt
```

```
## [1] 4.75
```

```
rmse_results <- rmse_results %>%
  add_row(method = paste0("Regularized movie + user (lambda = ", lambda_opt, ")"),
          RMSE = min(rmses))
rmse_results
```

method	RMSE
Just the average	1.0599043
Movie effect	0.9437429
Movie + user effect	0.8659319
Regularized movie + user (lambda = 4.75)	0.8652421

The best λ came out to 4.75, but honestly the improvement over the unregularized version is small (0.8659 to 0.8652). I think that's because the 10M dataset, unlike the small one used earlier in the course series, doesn't have that many movies or users with just one or two ratings - most of the really sparse cases got filtered out when the dataset was built. So regularization helps a little here, but it's not doing the heavy lifting some people might expect from the earlier course exercises. At this point the bias-based approach is close to its ceiling, which is the reason I didn't stop here.

2.6 Stage 3: matrix factorization

The problem with the bias model is that it can only capture “this movie is generally good” and “this user is generally harsh,” but not more specific patterns like “this user likes action movies more than average” or “this movie appeals disproportionately to users who also liked X.” Matrix factorization handles that by representing each movie and each user as a vector of latent factors, and predicting a rating as the dot product of the two - which lets the model pick up on these finer-grained interactions instead of just additive averages.

I used the `recoSystem` package, which implements this with stochastic gradient descent and is built to work directly with the kind of sparse user-item data we have here (it doesn’t need the full matrix in memory, which matters at this scale).

```
train_reco <- with(train_set, data_memory(user_index = userId,
                                          item_index = movieId,
                                          rating = rating))
test_reco <- with(test_set, data_memory(user_index = userId,
                                       item_index = movieId,
                                       rating = rating))
```

`recoSystem` ships with a `tune()` function that does a proper cross-validated grid search, but it turned out to be too slow to run comfortably on my machine (it refits several times per parameter combination). Instead I did a smaller manual grid search directly against `test_set`, which is less exhaustive but still gave me a good enough sense of which hyperparameters mattered:

```
grid <- expand.grid(dim = c(10, 20, 30),
                  costp_l2 = c(0.01, 0.1),
                  costq_l2 = c(0.01, 0.1))

grid_rmse <- numeric(nrow(grid))

for (i in seq_len(nrow(grid))) {
  set.seed(1)
  r <- Reco()
  r$train(train_reco, opts = list(dim = grid$dim[i],
                                  lrate = 0.1,
                                  costp_l2 = grid$costp_l2[i],
                                  costq_l2 = grid$costq_l2[i],
                                  nthread = 1,
                                  niter = 20,
                                  verbose = FALSE))

  pred <- r$predict(test_reco, out_memory())
  grid_rmse[i] <- RMSE(test_set$rating, pred)
}

grid$RMSE <- grid_rmse
grid %>% arrange(RMSE)
```

dim	costp_l2	costq_l2	RMSE
30	0.01	0.10	0.7906767
20	0.01	0.10	0.7948185
20	0.10	0.01	0.7971883
30	0.10	0.01	0.7992475
20	0.01	0.01	0.8039591
10	0.01	0.01	0.8148739
30	0.01	0.01	0.8159411

dim	costp_l2	costq_l2	RMSE
10	0.10	0.01	0.8164447
30	0.10	0.10	0.8214440
20	0.10	0.10	0.8231134
10	0.01	0.10	0.8255141
10	0.10	0.10	0.8341260

Higher `dim` (more latent factors) and a somewhat higher `costq_l2` (regularization on the movie factors) than on `costp_l2` (user factors) both helped. I nudged the best combination a bit further - more dimensions, more iterations - and landed on `dim = 40`, `costp_l2 = 0.01`, `costq_l2 = 0.10`, `niter = 30`:

```
set.seed(1)
r_test <- Reco()
r_test$train(train_reco, opts = list(dim = 40, lrate = 0.1,
                                     costp_l2 = 0.01, costq_l2 = 0.10,
                                     nthread = 1, niter = 30, verbose = FALSE))
pred_reco <- r_test$predict(test_reco, out_memory())
rmse_reco <- RMSE(test_set$rating, pred_reco)
rmse_reco

## [1] 0.7884909

rmse_results <- rmse_results %>%
  add_row(method = "Matrix factorization (recosystem)", RMSE = rmse_reco)
rmse_results
```

method	RMSE
Just the average	1.0599043
Movie effect	0.9437429
Movie + user effect	0.8659319
Regularized movie + user ($\lambda = 4.75$)	0.8652421
Matrix factorization (recosystem)	0.7884909

That's a solid jump down from the regularized bias model - about 0.079 off the RMSE, which is a bigger improvement than anything I got out of the bias-based approach.

I should mention: I ran this whole grid search with `nthread = 1` because that's what my hardware allowed at the time. If you're running this with more cores available, bump `nthread` up and you can afford a wider grid (more `dim` values in particular) without it taking forever - I'd expect a small further improvement from that, but I didn't have the chance to test it properly.

3 Results

With hyperparameters chosen using only `train_set`/`test_set`, I trained the final model on the *entire* `edx` set and evaluated it - for the first and only time - on `final_holdout_test`:

```
edx_reco <- with(edx, data_memory(user_index = userId,
                                 item_index = movieId,
                                 rating = rating))
final_reco <- with(final_holdout_test, data_memory(user_index = userId,
                                                    item_index = movieId,
                                                    rating = rating))
```

```

set.seed(1)
r_final <- Reco()
r_final$train(edx_reco, opts = list(dim = 40, lrate = 0.1,
                                   costp_l2 = 0.01, costq_l2 = 0.10,
                                   nthread = 1, niter = 30, verbose = FALSE))

predicted_ratings <- r_final$predict(final_reco, out_memory())
final_rmse <- RMSE(final_holdout_test$rating, predicted_ratings)
final_rmse

## [1] 0.7786509

rmse_results <- rmse_results %>%
  add_row(method = "FINAL MODEL - final_holdout_test", RMSE = final_rmse)
knitr::kable(rmse_results, digits = 5)

```

method	RMSE
Just the average	1.05990
Movie effect	0.94374
Movie + user effect	0.86593
Regularized movie + user (lambda = 4.75)	0.86524
Matrix factorization (recoSYSTEM)	0.78849
FINAL MODEL - final_holdout_test	0.77865

The final RMSE on `final_holdout_test` is **0.7787**, well below the 0.865 the bias-based model tops out at, and below the 0.86490 threshold that's usually held up as the top grading tier for this project.

4 Conclusion

Going from the mean-plus-bias approach to matrix factorization made a bigger difference than I expected going in - I assumed regularization would be doing more work, but on this dataset it's a marginal gain at best, while switching model families entirely is what actually moved the RMSE. That's probably the main thing I'm taking away from this: knowing when a model class has hit its ceiling, versus when it just needs more tuning, matters more than I gave it credit for before doing this project.

There's a real limitation in how I tuned the matrix factorization model. Ideally I'd have used proper k-fold cross-validation (which is what `recoSYSTEM::tune()` does) rather than a single train/test split and a fairly small manual grid, and I'd have searched over more values of `dim` in particular, since that was the parameter that seemed to matter most. I didn't do that here because of CPU constraints - single-threaded training on 9 million ratings is slow enough that a wide cross-validated grid search wasn't practical in the time I had. With more compute, I'd expect another small improvement from a more thorough search.

A few other things I'd try if I picked this back up: bringing in the `genres` field (currently completely unused - a genre-specific bias term, or interacting genre with user preferences, could pick up signal the current model misses), and using the `timestamp` column to model how a movie's perceived quality or a user's rating habits might drift over time. Both were out of scope for what I had time for here, but they're the two most obvious features sitting unused in the data.